The central place for defining and configuring security policies in Spring Security is a `SecurityWebFilterChain` bean. That object tells the framework which filters should be enabled. You can build a `SecurityWebFilterChain` bean through the DSL provided by `ServerHttpSecurity`.

For now, we want to comply with the following requirements:

- All endpoints exposed by Edge Service must require user authentication.
- The authentication must happen via a login form page.

To collect all the configurations related to security, create a `SecurityWebFilterChain` bean in a new `SecurityConfig` class (`com.polarbookshop.edgeservice.config` package):

```
@Bean          ❶
SecurityWebFilterChain springSecurityFilterChain(
  ServerHttpSecurity http
) {}
```

❶ The SecurityWebFilterChain bean is used to define and configure security policies for the application.

The `ServerHttpSecurity` object, autowired by Spring, provides a convenient DSL for configuring Spring Security and building a `SecurityWebFilterChain` bean. With `authorizeExchange()`, you can define access policies for any request (called an *exchange* in reactive Spring). In this case, we want all requests to require authentication (`authenticated()`):

```
@Bean
SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
```